



# AUDIT REPORT

---

May 2026

For



**KindWorld**

# Table of Content

|                                                                                                                    |    |
|--------------------------------------------------------------------------------------------------------------------|----|
| Executive Summary                                                                                                  | 04 |
| Number of Security Issues per Severity                                                                             | 06 |
| Summary of Issues                                                                                                  | 07 |
| Checked Vulnerabilities                                                                                            | 08 |
| Techniques and Methods                                                                                             | 09 |
| Types of Severity                                                                                                  | 11 |
| Types of Issues                                                                                                    | 12 |
| Severity Matrix                                                                                                    | 13 |
|  <b>Critical Severity Issues</b> | 14 |
| 1. Permanent Lockup of KNS Tokens                                                                                  | 14 |
|  <b>Medium Severity Issues</b>  | 16 |
| 2. Off-by-one error allows premature unstaking                                                                     | 16 |
| 3. Users may not have total distribution period time to claim their rewards                                        | 17 |
| 4. Valid last-second claims can be failed                                                                          | 18 |
| 5. Minimum 1 USDC claim requirement can prevent genuine users from claiming & enable dilution attacks              | 19 |
| 6. Users lose their total rewards for epochs where `emergency_withdraw()` was called                               | 21 |
|  <b>Low Severity Issues</b>     | 23 |
| 7. Initialization can be frontran                                                                                  | 23 |
| 8. Epoch PDA and its vault are not closed in close_epoch()                                                         | 24 |



|                                                                                                              |    |
|--------------------------------------------------------------------------------------------------------------|----|
|  <b>Informational Issues</b> | 26 |
| 9. Treasury's address can't be updated                                                                       | 26 |
| 10. Swap the position of balance check & merkle proof validation                                             | 27 |
| 11. Insufficient validations on merkle root during epoch initialization                                      | 28 |
| Automated Tests                                                                                              | 29 |
| Closing Summary & Disclaimer                                                                                 | 30 |



# Executive Summary

|                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project Name</b>       | KindWorld                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Protocol Type</b>      | Staking                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Project URL</b>        | <a href="https://docs.kindswap.world/">https://docs.kindswap.world/</a>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Overview</b>           | <p>Kindswapworld is Anchor-based Solana staking + reward distribution program that allows users to stake KNS tokens and earn USDC rewards via epoch-based distributions.</p> <ul style="list-style-type: none"><li>• Users can stake KNS with lock tiers (1–12 months), where longer locks give higher multipliers and increase their weighted stake used for reward calculations.</li><li>• The protocol tracks a global weighted stake and enforces one active stake per wallet, with tokens held securely in PDA-controlled vaults.</li><li>• Rewards are distributed in epochs funded by the admin, where each epoch uses a Merkle root (off-chain computed) to define user reward allocations.</li><li>• Users claim USDC rewards permissionlessly by submitting a valid Merkle proof, with protections like duplicate-claim prevention and proof size limits.</li><li>• After the claim period, unclaimed rewards are swept to treasury, with additional admin controls like pause, emergency withdrawal, and config updates.</li></ul> <p>Overall, the design combines on-chain staking + off-chain reward computation, optimized for scalability and gas efficiency using Merkle proofs.</p> |
| <b>Audit Scope</b>        | The scope of the Audit is to analyse the security, correctness and code quality of KindWorld Staking Contract.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Source Code link</b>   | <a href="https://github.com/KindWorldLabs/Kns_Staking-Contract/blob/main/app/programs/staking/src/lib.rs">https://github.com/KindWorldLabs/Kns_Staking-Contract/blob/main/app/programs/staking/src/lib.rs</a>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Contracts in Scope</b> | app/programs/staking/src/lib.rs                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Commit Hash</b>        | aeb61b10679f06465eae3e42ce5dfeee48355ebc                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Language</b>           | Rust                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Blockchain</b>         | Solana                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |



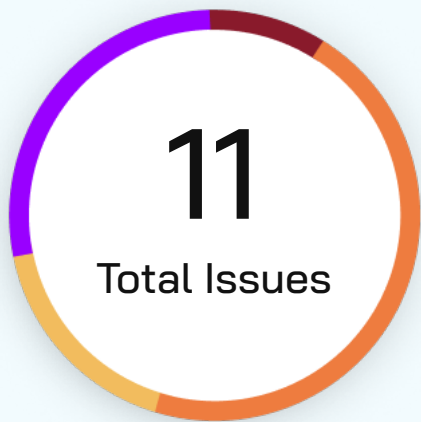
|                              |                                          |
|------------------------------|------------------------------------------|
| <b>Review 1</b>              | April 23, 2026 - April 29, 2026          |
| <b>Updated Code Received</b> | April 30th 2026                          |
| <b>Review 2</b>              | April 30th, 2026 - May 4, 2026           |
| <b>Fixed In</b>              | a4aa2aecff5e5961d1e091aae84aa31e2b8ab56a |

**Verify the Authenticity of Report on QuillAudits Leaderboard:**

<https://www.quillaudits.com/leaderboard>



# Number of Issues per Severity



|               |            |
|---------------|------------|
| Critical      | 1 (9.09%)  |
| High          | 0 (0.0%)   |
| Medium        | 5 (45.45%) |
| Low           | 2 (18.18%) |
| Informational | 3 (27.27%) |

|        |                    | Severity |      |        |     |               |
|--------|--------------------|----------|------|--------|-----|---------------|
|        |                    | Critical | High | Medium | Low | Informational |
| Issues | Open               | 0        | 0    | 0      | 0   | 0             |
|        | Acknowledged       | 0        | 0    | 0      | 0   | 0             |
|        | Partially Resolved | 0        | 0    | 0      | 0   | 0             |
|        | Resolved           | 1        | 0    | 5      | 2   | 3             |



# Summary of Issues

| Issue No. | Issue Title                                       | Severity      | Status   |
|-----------|---------------------------------------------------|---------------|----------|
| 1         | Permanent lockup of KNS tokens                    | Critical      | Resolved |
| 2         | Off-by-one error allows premature unstaking       | Medium        | Resolved |
| 3         | Reduced claim window due to deadline misplacement | Medium        | Resolved |
| 4         | Last-second claims may fail                       | Medium        | Resolved |
| 5         | Minimum claim threshold causes reward loss        | Medium        | Resolved |
| 6         | Emergency withdraw blocks claims permanently      | Medium        | Resolved |
| 7         | Initialization can be frontrun                    | Low           | Resolved |
| 8         | Epoch and vault accounts not closed               | Low           | Resolved |
| 9         | Treasury address cannot be updated                | Informational | Resolved |
| 10        | Inefficient merkle verification order             | Informational | Resolved |
| 11        | Missing merkle root validation                    | Informational | Resolved |



# Checked Vulnerabilities

We have scanned the solana program for commonly known and more specific vulnerabilities. Here are some of the commonly known vulnerabilities that we considered:

- ✓ Signer authorization
- ✓ Account data matching
- ✓ Sysvar address checking
- ✓ Owner checks
- ✓ Type cosplay
- ✓ Initialization
- ✓ Arbitrary cpi
- ✓ Duplicate mutable accounts
- ✓ Bump seed canonicalization
- ✓ PDA Sharing
- ✓ Incorrect closing accounts
- ✓ Missing rent exemption checks
- ✓ Arithmetic overflows/underflows
- ✓ Numerical precision errors
- ✓ Solana account confusions
- ✓ Casting truncation
- ✓ Insufficient SPL token account verification
- ✓ Signed invocation of unverified programs





# Techniques and Methods

**Throughout the audit of Solana Programs, care was taken to ensure:**

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

**The following techniques, methods, and tools were used to review all the smart contracts:**

## Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

## Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



### Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

### Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.



# Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

## ■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

## ■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

## ■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

## ■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

## ■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



# Types of Issues

## Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

## Resolved

These are the issues identified in the initial audit and have been successfully fixed.

## Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

## Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.



# Severity Matrix

|            |        | Impact   |        |        |
|------------|--------|----------|--------|--------|
|            |        | High     | Medium | Low    |
| Likelihood | High   | Critical | High   | Medium |
|            | Medium | High     | Medium | Low    |
|            | Low    | Medium   | Low    | Low    |

## Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

## Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



# Critical Severity Issues

## Permanent Lockup of KNS Tokens

**Resolved**

### Description

Users call `unstake()` once their lock period has been ended, the `unstake()` function attempts to close the stake vault token account, which requires the account balance to be exactly zero. An attacker can send additional tokens or lamports to the stake vault, preventing the `close\_account` operation from succeeding and making it impossible for legitimate users to unstake.

```
1  ```rust
2  // Line 372-378 - Closing vault that can be made non-empty
3  token::close_account(CpiContext::new_with_signer(
4      ctx.accounts.token_program.to_account_info(),
5      CloseAccount {
6          account: ctx.accounts.stake_vault.to_account_info(),
7          destination: ctx.accounts.owner.to_account_info(),
8          authority: ctx.accounts.config.to_account_info(),
9      },
10     signer_seeds,
11 ))?;
12  ```
```

The Solana token program's `CloseAccount` instruction requires the account to have exactly 0 tokens. If an attacker sends even 1 token to the vault, the transaction will fail with an error like "close\_account: account still has balances".

### Impact

- Users become locked into their positions indefinitely after the lock period expires
- Users cannot recover their staked KNS tokens
- This creates a griefing attack where an attacker can DOS multiple users' unstake operations

### Remediation

Before closing the stake vault, transfer any remaining balance to the owner or burn it. Alternatively, withdraw all tokens first and don't rely on the account being exactly empty:



```
1  ```rust
2  // Transfer any remaining tokens (including dust) to owner
3  let remaining = ctx.accounts.stake_vault.amount;
4  if remaining > 0 {
5      token::transfer(
6          CpiContext::new_with_signer(
7              ctx.accounts.token_program.to_account_info(),
8              Transfer {
9                  from: ctx.accounts.stake_vault.to_account_info(),
10                 to: ctx.accounts.owner_kns_ata.to_account_info(),
11                 authority: ctx.accounts.config.to_account_info(),
12             },
13             signer_seeds,
14         ),
15         remaining,
16     )?;
17 }
18
19 // Now safe to close
20 token::close_account(...)?;
21 ```
```

# Medium Severity Issues

## Off-by-one error allows premature unstaking

**Resolved**

### Description

The `unstake()` function uses `now >= lock\_end\_ts` to check if the lock period has expired. This should be `now > lock\_end\_ts` to ensure users can only un stake AFTER the lock expires, not exactly AT the expiry timestamp.

```
1  ```rust
2  // Line 334-335 - Should use > instead of >=
3  let now = Clock::get()?.unix_timestamp;
4  require!(now >= stake_state.lock_end_ts, StakingError::StillLocked);
5  ```
```

If a user locks for exactly 1 month (2,592,000 seconds), they should only un stake after that period passes. With `>=`, they can un stake at the exact moment the lock expires, which is technically part of the lock period. This will be problematic because rewards are calculated offchain based on snapshots of users' weights, if someone unstakes and their balance had already been counted towards epoch rewards, it would lead to users get less rewards for this epoch than intended.

### Impact

Users can un stake slightly before the intended lock period ends

### Recommendation

Change the comparison operator from `>=` to `>`:

```
1  ```rust
2  // Line 334-335 - Fixed version
3  let now = Clock::get()?.unix_timestamp;
4  require!(now > stake_state.lock_end_ts, StakingError::StillLocked);
5  ```
```

This ensures users can only un stake strictly after the lock period expires.





## Users may not have total distribution period time to claim their rewards

**Resolved**

### Description

The `initialize_epoch()` function sets the `claim_deadline` based on the current time plus the distribution period. However, funding can happen much later via `fund_epoch()`. This creates a scenario where the claim window is shortening while the epoch remains unfunded.

```
1  ```rust
2  // Line 409-413 - Deadline calculated at initialize_epoch
3  let claim_deadline = now
4      .checked_add(cfg.distribution_period)
5      .ok_or(StakingError::MathOverflow)?;
6
7  // But funding happens later in fund_epoch (line ~468)
8  ```
```

### Example

- Admin initializes epoch on Day 1 → deadline = Day 1 + 14 days = Day 15
- Admin funds epoch on Day 13 → users have only 2 days to claim instead of the intended 14 days

### Impact

- Users get significantly less time to claim rewards than intended
- If funding is delayed for any reason, users lose the ability to claim entire epochs
- Creates confusion about the actual claim window duration

### Recommendation

Move the deadline calculation from `initialize_epoch()` to `fund_epoch()`:

```
1  ```rust
2  // In fund_epoch() - calculate deadline when funding, not when initializing
3  pub fn fund_epoch(ctx: Context<FundEpoch>, _epoch_id: u64, amount: u64) -> Result<()> {
4      // ... existing checks ...
5
6      let epoch = &mut ctx.accounts.epoch;
7      let now = Clock::get()?.unix_timestamp;
8
9      // Set claim_deadline at funding time
10     let cfg = &ctx.accounts.config;
11     epoch.claim_deadline = now
12         .checked_add(cfg.distribution_period)
13         .ok_or(StakingError::MathOverflow)?;
14
15     // ... rest of function ...
16 }
17 ```
18
```

Remove the deadline calculation from `initialize_epoch()`.



## Valid last-second claims can be failed

**Resolved**

### Description

The `close_epoch()` function can be called when `now >= claim_deadline`. Users can still claim when `now <= claim_deadline`. This creates a condition where if the admin calls `close_epoch()` at the exact moment the deadline expires, legitimate users attempting to claim at the same moment may fail.

```
1  ```rust
2  // In close_epoch() - Line 715
3  require!(
4      now >= epoch.claim_deadline,
5      StakingError::DeadlineNotReached
6  );
7
8  // In claim() - Line 757
9  require!(
10     Clock::get()?.unix_timestamp <= epoch.claim_deadline,
11     StakingError::EpochExpired
12 );
13 ```
```

Both conditions use the same timestamp value with `==` boundary. If a block's timestamp is exactly at the deadline:

- `close_epoch()` will pass (`now >= deadline` is true)
- `claim()` will pass (`now <= deadline` is true)
- But if `close_epoch()` is executed first, it sets status to `Closing`, which causes `claim()` to fail

### Impact

- Users lose rewards through no fault of their own due to bad timing
- Creates incentive for admin to DoS users by closing at the deadline
- Users cannot reliably claim rewards near deadlines

### Recommendation

Use strict inequality for admin operations(strict inequality for `close_epoch`):

```
1  ```rust
2  // Option 1: Use strict inequality for close_epoch
3  pub fn close_epoch(ctx: Context<CloseEpoch>, epoch_id: u64) -> Result<> {
4      // ... existing checks ...
5
6      let now = Clock::get()?.unix_timestamp;
7      require!(
8          now > epoch.claim_deadline, // Changed from >= to >
9          StakingError::DeadlineNotReached
10 );
11
12     // ... rest of function ...
13 }
14 ```
```



## Minimum 1 USDC claim requirement can prevent genuine users from claiming & enable dilution attacks

**Resolved**

### Description

The `claim()` function requires a minimum claim amount of 1 USDC (1,000,000 in 6-decimal precision). Users with small stakes relative to the total weighted stake may earn less than 1 USDC per epoch and are unable to claim their rewards. Since rewards are not aggregated and the total weight of KNS will always be increasing which in turn will dilute the rewards even more, which means user gets no benefit from staking on top of that his KNS tokens keep locked for his chosen lock tier, then these rewards are swept to treasury on `close_epoch()`, permanently lost to users. In case user was earning 0.9 USDC per epoch and there are 15 epochs per month, user essentially loses 13 USDC.

```
1  ``rust
2  // Line 168 - Minimum claim constant
3  const USDC_MIN_CLAIM: u64 = 1_000_000;
4
5  // Line 758 - Enforced in claim()
6  require!(amount >= USDC_MIN_CLAIM, StakingError::BelowMinClaim);
7  ``
```

Not only that, anyone who knows there is a threshold on claim can create fake positions and inflate the weighted amount (the `total_weighted_stake`) without ever unstaking, reducing other users' reward allocations and forcing them below the 1 USDC minimum.

A malicious actor can:

1. Stake an enormous amount of KNS for 12 months (3× multiplier)
2. Wait for epoch rewards to be calculated based on the inflated `total_weighted_stake`
3. Never un stake, keeping the stake active
4. Repeatedly dilute the pool by staking more and more & prevent small stakers from claiming their rewards

### Example Scenario

- Epoch has 10,000 USDC in rewards
- Alice stakes 1 KNS for 1 month (1× multiplier) = 1 weighted unit
- Bob stakes 1,000,000 KNS for 12 months (3× multiplier) = 3,000,000 weighted units
- Total weighted stake = 3,000,001
- Alice's share:  $10,000 \times (1 / 3,000,001) \approx 0.0033$  USDC → cannot claim, forfeits rewards

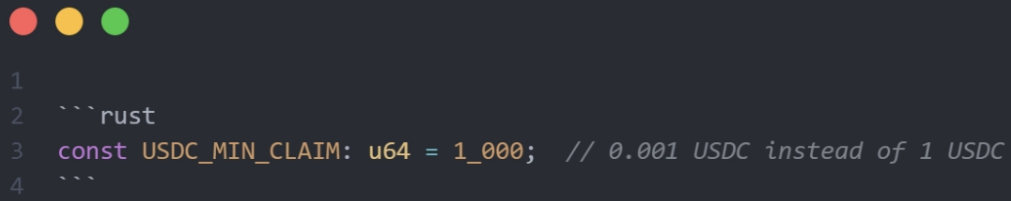
### Impact

- Small stakeholders permanently lose earned rewards
- Rewards system becomes inequitable—only large stakers can claim
- Attacker can perform "dilution DOS" by inflating total stake and forcing small users below 1 USDC threshold
- Treasury accrues unclaimed rewards that were meant for users



### Recommendation

Remove or significantly lower the minimum:



```
1  
2 ```rust  
3 const USDC_MIN_CLAIM: u64 = 1_000; // 0.001 USDC instead of 1 USDC  
4 ```
```

## Users lose their total rewards for epochs where `emergency\_withdraw()` was called

**Resolved**

### Description

The `emergency\_withdraw()` function is intended for incident response to drain an epoch vault. However, it sets the epoch status to `EpochStatus::Closed`, which prevents users from claiming their rightful rewards even after the emergency is resolved.

```
1  ```rust
2  // Line 827-828 - Sets status to Closed immediately
3  ctx.accounts.epoch.status = EpochStatus::Closed;
4
5  // This prevents ALL future claims for this epoch
6  ```
```

Once `emergency\_withdraw()` is executed, the epoch transitions to `Closed` status. The `claim()` function checks:

```
1  ```rust
2  // Line 757 - claim() requires Active status
3  require!(
4      epoch.status == EpochStatus::Active,
5      StakingError::InvalidEpochStatus
6  );
7  ```
```

Users can no longer claim their rewards even if the emergency is resolved.

### Impact

- Emergency operations permanently disable user claims for that epoch
- Users lose access to their rightful rewards indefinitely
- No recovery mechanism after emergency\_withdraw executes

### Recommendation

Option 1: Don't close the epoch in emergency\_withdraw, only transfer funds:



```

1  ```rust
2  pub fn emergency_withdraw(ctx: Context<EmergencyWithdraw>, epoch_id: u64) -> Result<> {
3      let cfg = &ctx.accounts.config;
4      require_admin(cfg, &ctx.accounts.admin)?;
5
6      let vault_balance = ctx.accounts.epoch_vault.amount;
7      require!(vault_balance > 0, StakingError::InvalidAmount);
8
9      // Transfer to treasury
10     let epoch_id_bytes = epoch_id.to_le_bytes();
11     let epoch_bump = ctx.bumps.epoch;
12     let signer_seeds: &[&[u8]] = &[&[b"epoch", epoch_id_bytes.as_ref(), &[epoch_bump]]];
13
14     token::transfer(
15         CpiContext::new_with_signer(
16             ctx.accounts.token_program.to_account_info(),
17             Transfer {
18                 from: ctx.accounts.epoch_vault.to_account_info(),
19                 to: ctx.accounts.treasury_usdc_ata.to_account_info(),
20                 authority: ctx.accounts.epoch.to_account_info(),
21             },
22             signer_seeds,
23         ),
24         vault_balance,
25     )?;
26
27     // DON'T set status to Closed-Leave it as Active for claims
28     // Status can be manually set back to Active via admin function if needed
29
30     emit!(EmergencyWithdrawExecuted {
31         epoch_id,
32         amount: vault_balance,
33         timestamp: Clock::get()?.unix_timestamp,
34     });
35
36     Ok(())
37 }
38 ```

```

Option 2: Add a separate function to recover/restore the epoch after emergency\_withdraw:

```

1  ```rust
2  pub fn restore_epoch_after_emergency(
3      ctx: Context<UpdateConfig>,
4      epoch_id: u64,
5  ) -> Result<> {
6      let cfg = &ctx.accounts.config;
7      require_admin(cfg, &ctx.accounts.admin)?;
8
9      ctx.accounts.epoch.status = EpochStatus::Active;
10
11     emit!(EpochRestored { epoch_id });
12     Ok(())
13 }
14 ```

```

# Low Severity Issues

## Initialization can be frontran

**Resolved**

### Description

The `initialize()` function has no access control mechanism. Any user can call it before the intended admin, allowing them to set arbitrary values for the admin, mints, and treasury addresses.

```
1  ```rust
2  // Line 198-215 - No access control on initialize
3  pub fn initialize(
4      ctx: Context<Initialize>,
5      admin: Pubkey,
6      kns_mint: Pubkey,
7      usdc_mint: Pubkey,
8      treasury: Pubkey,
9  ) -> Result<()> {
10     let cfg = &mut ctx.accounts.config;
11     cfg.admin = admin; // Can be set to attacker's address
12     cfg.kns_mint = kns_mint;
13     cfg.usdc_mint = usdc_mint;
14     cfg.treasury = treasury; // Can redirect funds
15     // ...
16 }
17 ...
```

### Recommendation

Add a signer check to ensure only an authorized deployer can call `initialize()`:

```
1  ```rust
2  pub fn initialize(
3      ctx: Context<Initialize>,
4      admin: Pubkey,
5      kns_mint: Pubkey,
6      usdc_mint: Pubkey,
7      treasury: Pubkey,
8  ) -> Result<()> {
9      // Only allow a predefined deployer to initialize
10     require!(
11         ctx.accounts.payer.key() == AUTHORIZED_DEPLOYER,
12         StakingError::Unauthorized
13     );
14
15     let cfg = &mut ctx.accounts.config;
16     // ... rest of function
17 }
18 ...
```



## Epoch PDA and its vault are not closed in close\_epoch()

**Resolved**

### Description

The `close\_epoch()` function transfers unclaimed USDC to treasury but does not close the epoch vault token account or the epoch PDA. This creates orphaned accounts that consume rent-exempt balances indefinitely.

```
1
2  ```rust
3  // Line 699-723 - Vault transferred but not closed
4  if vault_balance > 0 {
5      // ... transfer to treasury ...
6  }
7
8  // After this, epoch_vault TokenAccount still exists (not closed)
9  // and epoch PDA still exists (not closed)
10
11  ctx.accounts.epoch.status = EpochStatus::Closed;
12  // Status updated but account not closed
13
14  emit!(EpochClosed { ... });
15  // No close_account instruction for vault
16  ```
```

After an epoch is closed, these accounts are no longer needed but remain on-chain, wasting blockchain resources.

### Impact

- Rent-exempt balances accumulate over many epochs
- Wastes Solana network resources

### Recommendation

Close both the token account and the epoch PDA after sweeping:

```
1
2  ```rust
3  // After transferring funds
4  ctx.accounts.epoch.status = EpochStatus::Closed;
5
6  // Close the vault token account
7  token::close_account(CpiContext::new_with_signer(
8      ctx.accounts.token_program.to_account_info(),
9      CloseAccount {
10          account: ctx.accounts.epoch_vault.to_account_info(),
11          destination: ctx.accounts.admin.to_account_info(),
12          authority: ctx.accounts.epoch.to_account_info(),
13      },
14      signer_seeds,
15  ))?;
```





Add `close = admin` to the epoch account in the `CloseEpoch` context:

```
1  ```rust
2  #[derive(Accounts)]
3  pub struct CloseEpoch<'info> {
4      // ...
5      #[account(
6          mut,
7          close = admin, // Add this line to close the account
8          seeds = [b"epoch", epoch_id.to_le_bytes().as_ref()],
9          bump
10     )]
11     pub epoch: Account<'info, Epoch>,
12     // ...
13 }
14 ```
15
```

# Informational Issues

## Treasury's address can't be updated

**Resolved**

### Description

The treasury address is set once during `initialize()` and cannot be changed. There is no `update\_treasury()` function, making it impossible to migrate the treasury address if the original address is compromised or needs to be updated.

```
1  ```rust
2  // Line 210 - Treasury is set once and immutable
3  cfg.treasury = treasury;
4
5  // No function exists to update it
6  ```
```

If the treasury private key is compromised or the address needs to be changed for operational reasons, the protocol cannot adapt.

### Impact

- If the treasury address is compromised, there's no way to redirect future sweeps
- Operational inflexibility—cannot update treasury even for legitimate reasons
- Unclaimed rewards will sweep to a potentially compromised address indefinitely

### Recommendation

Add an admin-controlled function to update the treasury address:

```
1  ```rust
2  pub fn update_treasury(ctx: Context<UpdateConfig>, new_treasury: Pubkey) -> Result<()> {
3      let cfg = &mut ctx.accounts.config;
4      require_admin(cfg, &ctx.accounts.admin)?;
5
6      let old_treasury = cfg.treasury;
7      cfg.treasury = new_treasury;
8
9      emit!(TreasuryUpdated {
10         old_treasury,
11         new_treasury,
12         timestamp: Clock::get()?.unix_timestamp,
13     });
14     Ok(())
15 }
16 ```
```



## Swap the position of balance check & merkle proof validation

**Resolved**

### Description

The `claim()` function verifies the Merkle proof before checking if the vault has sufficient USDC balance. Computing the Merkle root requires expensive hashing operations that consume significant compute units (CU). If the vault is empty, this computation is wasted.

```
1  ```rust
2  // Line 754-763 - Merkle verification happens first (expensive)
3  let leaf = compute_leaf(epoch_id, &claimer, amount);
4  require!(
5      verify_merkle_proof(&merkle_proof, epoch.merkle_root, leaf),
6      StakingError::InvalidMerkleProof
7  );
8
9  // Line 766-770 - Vault check happens after (should be before)
10 require!(
11     ctx.accounts.epoch_vault.amount >= amount,
12     StakingError::InsufficientFunds
13 );
14 ```
```

The vault balance check is a simple comparison and should happen first to fail fast.

### Impact

- Wasted compute units on invalid claims
- Higher transaction costs for users

### Recommendation

Reorder the checks to do the cheap vault balance check first:

```
1  ```rust
2  pub fn claim(
3      ctx: Context<Claim>,
4      epoch_id: u64,
5      amount: u64,
6      merkle_proof: Vec<u8; 32>,
7  ) -> Result<> {
8      // ... existing checks (paused, mint, ATA, epoch status, deadline, min amount) ...
9
10     // NEW: Check vault balance first (cheap operation)
11     require!(
12         ctx.accounts.epoch_vault.amount >= amount,
13         StakingError::InsufficientFunds
14     );
15
16     // NOW: Do expensive Merkle proof verification
17     let claimer = ctx.accounts.owner.key();
18     let leaf = compute_leaf(epoch_id, &claimer, amount);
19     require!(
20         verify_merkle_proof(&merkle_proof, epoch.merkle_root, leaf),
21         StakingError::InvalidMerkleProof
22     );
23
24     // ... rest of function ...
25 }
26 ```
27
```



## Insufficient validations on merkle root during epoch initialization

**Resolved**

### Description

The `initialize_epoch()` function accepts a `merkle_root` parameter without validating that it is not all zeros or otherwise invalid. An admin could accidentally or maliciously set the `merkle_root` to all zeros or another trivial value, breaking the entire epoch's reward distribution.

```
1  ```rust
2  // Line 423 - No validation on merkle_root
3  epoch.merkle_root = merkle_root;
4  // @audit make sure not set to all zeroes
5  ```
```

### Impact

- The entire epoch reward distribution can be exploited if the `merkle_root` is invalid

### Recommendation

Add validation to reject zero or otherwise suspicious merkle roots:

```
1  ```rust
2  pub fn initialize_epoch(
3      ctx: Context<InitializeEpoch>,
4      epoch_id: u64,
5      merkle_root: [u8; 32],
6      total_rewards: u64,
7      snapshot_ts: i64,
8  ) -> Result<()> {
9      require!(total_rewards > 0, StakingError::InvalidAmount);
10
11      // NEW: Validate merkle_root is not all zeros
12      let zero_root = [0u8; 32];
13      require!(merkle_root != zero_root, StakingError::InvalidMerkleRoot);
14
15      // Optional: Validate root is not some other trivial value
16      // (all 0xFF, etc.)
17
18      let cfg = &ctx.accounts.config;
19      require_admin(cfg, &ctx.accounts.admin)?;
20
21      // ... rest of function ...
22  }
23  ```
24
25  Add error variant:
26
27  ```rust
28  #[error_code]
29  pub enum StakingError {
30      // ... existing errors ...
31      #[msg("Invalid merkle root (all zeros or invalid)")]
32      InvalidMerkleRoot,
33  }
34  ```
35
36  ---
```



# Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



# Closing Summary

In this report, we have considered the security of KindWorld. We performed our audit according to the procedure described above.

Multiple issues, Critical, Medium, low and Informational Issues found during the Audit. KindWorld resolved all the issues mentioned above

# Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



# About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With eight years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



|                                 |                                     |
|---------------------------------|-------------------------------------|
| <b>8+</b><br>Years of Expertise | <b>1M+</b><br>Lines of Code Audited |
| <b>50+</b><br>Chains Supported  | <b>1500+</b><br>Projects Secured    |

## Follow Our Journey



# AUDIT REPORT

---

May 2026

For



**KindWorld**



Canada, India, Singapore, UAE, UK

[www.quillaudits.com](http://www.quillaudits.com)

[audits@quillaudits.com](mailto:audits@quillaudits.com)